

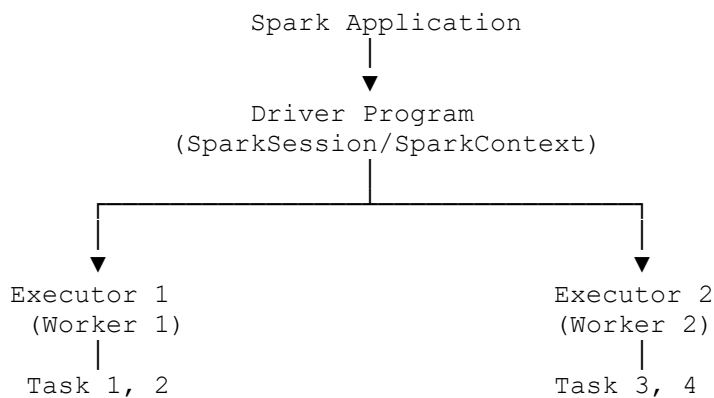
Driver vs. Executor in Apache Spark

The **Driver** and **Executors** are the two most important components in Spark architecture.

- The **Driver** is the **brain** of the Spark application. It plans, coordinates, and monitors the execution.
- The **Executors** are the **workers** that perform the actual data processing.

Together, they enable Spark to process data in parallel across a cluster.

Spark Architecture



What is the Driver?

The **Driver** is the central coordinator of a Spark application.

There is **one Driver per Spark application**.

It runs your Spark program and controls the entire execution process.

Responsibilities of the Driver

- Creates the **SparkSession** (and internally the **SparkContext**).
- Reads and understands your Spark code.
- Builds the **logical execution plan**.
- Optimizes the plan (using **Catalyst** for DataFrame/SQL operations).
- Creates the **DAG (Directed Acyclic Graph)**.
- Splits the DAG into **stages**.

- Splits stages into **tasks** (one task per partition).
 - Requests resources from the Cluster Manager.
 - Sends tasks to executors.
 - Monitors task execution.
 - Handles task failures and retries.
 - Collects results from executors.
-

What is an Executor?

An **Executor** is a JVM process that runs on a **Worker Node**.

Executors perform the actual computations on the data.

Each Spark application gets its own executors, which are terminated when the application finishes.

Responsibilities of an Executor

- Execute tasks assigned by the Driver.
 - Read data from HDFS, S3, databases, or other storage systems.
 - Perform transformations and actions.
 - Cache data in memory for faster reuse.
 - Write output to storage.
 - Report task status and results back to the Driver.
-

How the Driver and Executors Work Together

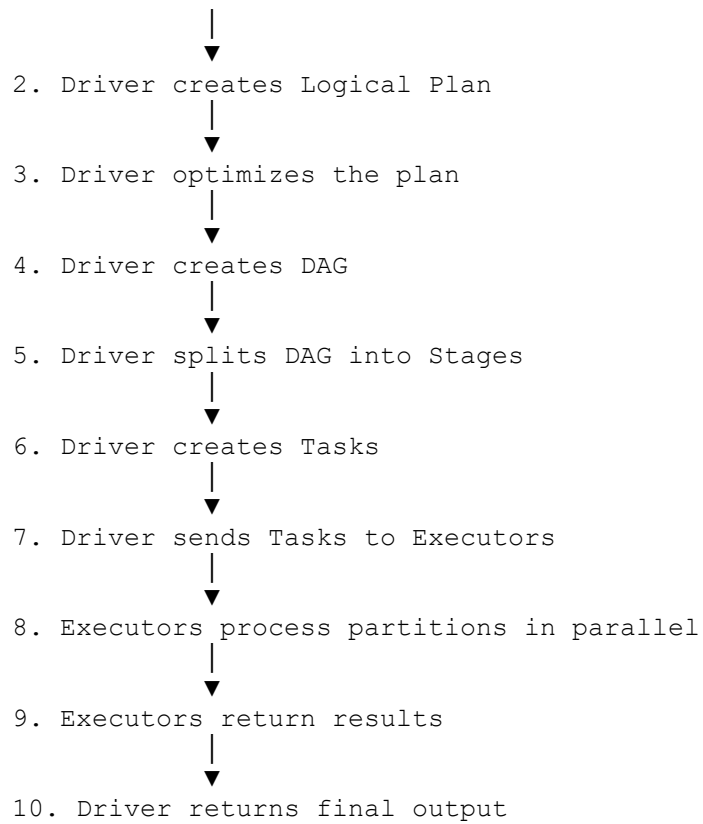
Suppose you run:

```
df = spark.read.csv("employees.csv", header=True)

df.filter(df.salary > 5000) \
  .groupBy("department") \
  .count() \
  .show()
```

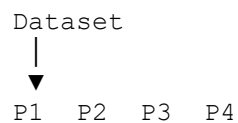
Execution flow:

1. Driver reads your code



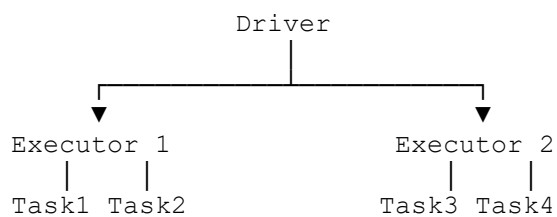
Example

Suppose your DataFrame has **4 partitions**.



The Driver creates **4 tasks**.

Execution:



Each executor processes its assigned partitions in parallel.

Driver vs. Executor

Feature	Driver	Executor
Role	Coordinates the application	Performs computations
Runs On	Driver node	Worker nodes
Number	One per Spark application	One or more per application
Creates SparkSession	✓ Yes	✗ No
Creates DAG	✓ Yes	✗ No
Creates Stages	✓ Yes	✗ No
Creates Tasks	✓ Yes	✗ No
Executes Tasks	✗ No	✓ Yes
Reads Data	Coordinates reading	Reads assigned partitions
Caches Data	✗ No	✓ Yes
Returns Results	Receives results	Sends results to Driver

Real-World Example

Imagine you need to process **1 TB** of sales data.

1. The **Driver** analyzes your Spark code and creates the execution plan.
 2. It divides the work into stages and tasks.
 3. The **Cluster Manager** launches executors on worker nodes.
 4. The Driver assigns tasks to the executors.
 5. Each executor processes its assigned partitions in parallel.
 6. Executors send results back to the Driver.
 7. The Driver combines the results and returns the final output.
-

Common Interview Questions

Q1. What is the Driver in Spark?

The **Driver** is the central coordinator of a Spark application. It creates the execution plan, builds the DAG, divides it into stages and tasks, schedules work, monitors execution, and collects results.

Q2. What is an Executor?

An **Executor** is a JVM process running on a worker node that executes tasks, processes data partitions, caches data in memory, and reports results back to the Driver.

Q3. Can a Spark application have multiple Drivers?

No.

A Spark application has **one Driver**, but it can have **multiple Executors**.

Q4. Who creates Tasks?

The **Driver** creates tasks after dividing the DAG into stages.

Q5. Who executes Tasks?

The **Executors** execute the tasks assigned by the Driver.

Q6. Does the Driver process data?

The Driver's primary role is **coordination**, not large-scale data processing. It plans and schedules execution, while **Executors** process the data. However, the Driver may handle small amounts of data returned by actions such as `collect()` or `take()`. Using `collect()` on very large datasets can cause the Driver to run out of memory.

Interview Summary

- The **Driver** is the **brain** of a Spark application. It creates the `SparkSession`, builds and optimizes the execution plan, creates the DAG, divides it into stages and tasks, requests resources, schedules execution, monitors progress, and collects results.

- **Executors** are JVM processes running on worker nodes. They execute tasks, process data partitions, cache intermediate data in memory, write output, and report status back to the Driver.
- A Spark application has **one Driver** and **one or more Executors**.
- The Driver **plans and coordinates** the work, while the Executors **perform the actual computations** in parallel.